Rust in Production

outline

https://doc.rust-lang.org/nomicon/ffi.html

http://reddit.com/r/rust/comments/90s2no/rust_c_interop/

https://eshard.com/posts/Rust-Cxx-interop

https://www.youtube.com/watch?v=ZbObQYEzZ3Y&list=PL2b0df3jKKiRFEuVNk76ufXaqOqEJ9sBZ&index=22

https://engineering.fb.com/2022/07/27/developer-tools/programming-languages-endorsed-for-server-side-use-at-meta/

govertment notice thing

https://dropbox.tech/infrastructure/rewriting-the-heart-of-our-sync-engine

quiz

final in 5 days

end of quarter feedback

bob bandes

marcin project

pika

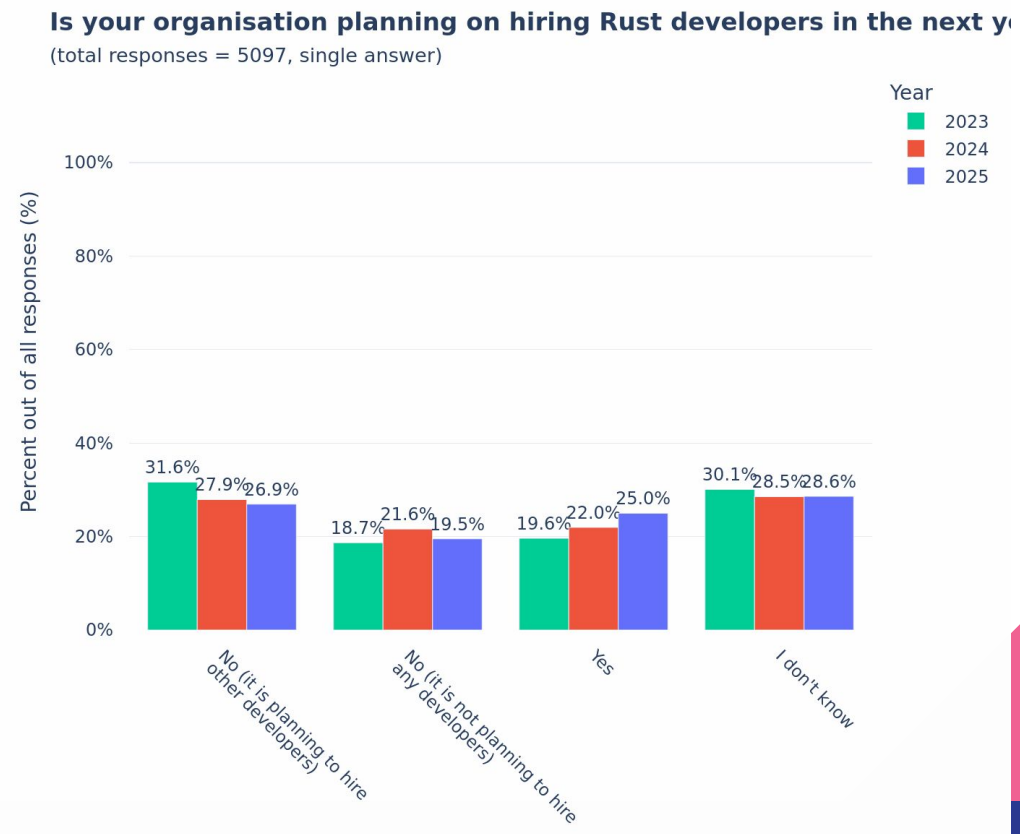
Administrivia

- HW9 due tonight
- End-of-Quarter Survey
- Bob Bandes Outstanding TA Award
- Final in 5 days
- Quiz at EOD

Rust in Production: Agenda

- Meta Analysis
- How to make it work
- Code Examples

Rust in Production: A Meta Analysis



FEBRUARY 26, 2024

Press Release: Future Software Should Be Memory Safe

Leaders in Industry Support White House Call to Address Root Cause of Many of the Worst Cyber Attacks

Read the full report here ↗

WASHINGTON – Today, the White House Office of the National Cyber Director (ONCD) released a report calling on the technical community to proactively reduce the attack surface in cyberspace. ONCD makes the case that technology manufacturers can prevent entire classes of vulnerabilities from entering the digital ecosystem by adopting memory safe programming languages. ONCD is also encouraging the research community to address the problem of software measurability to enable the development of better diagnostics that measure cybersecurity quality.

The report is titled *"Back to the Building Blocks: A Path Toward Secure and Measurable Software."* ↗

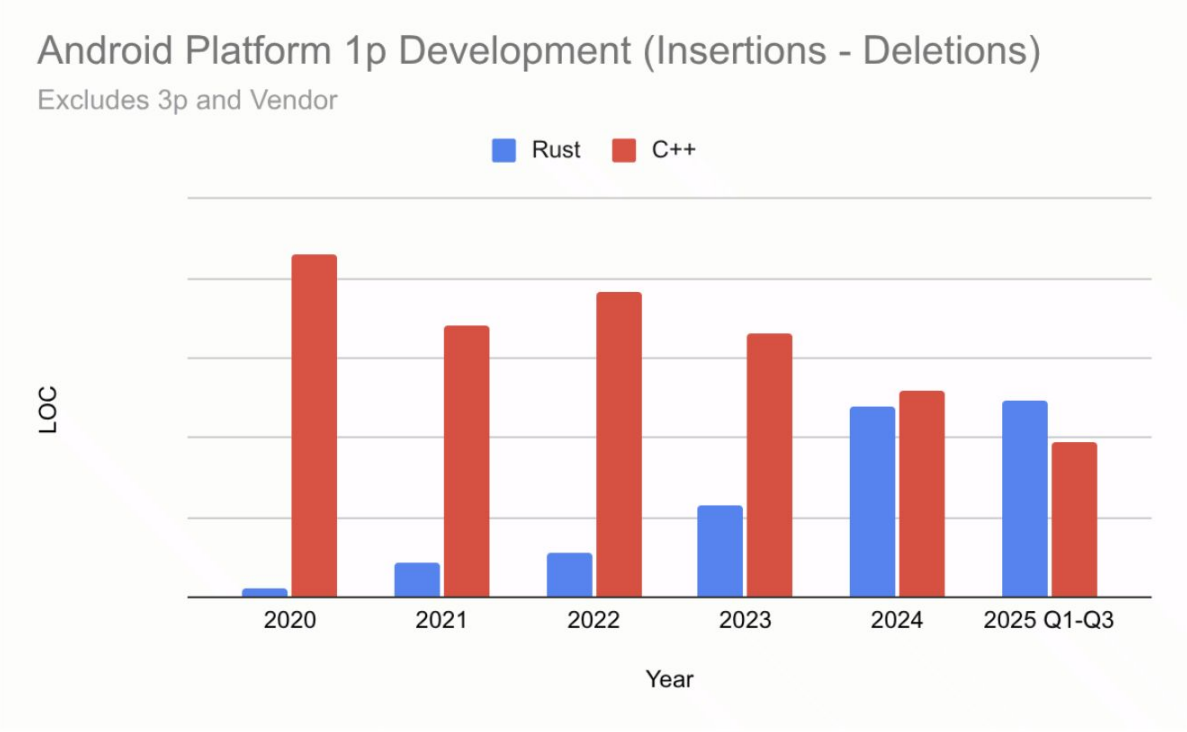
Google Security Blog

The latest news and insights from Google on security and safety on the Internet

Rust in Android: move fast and fix things

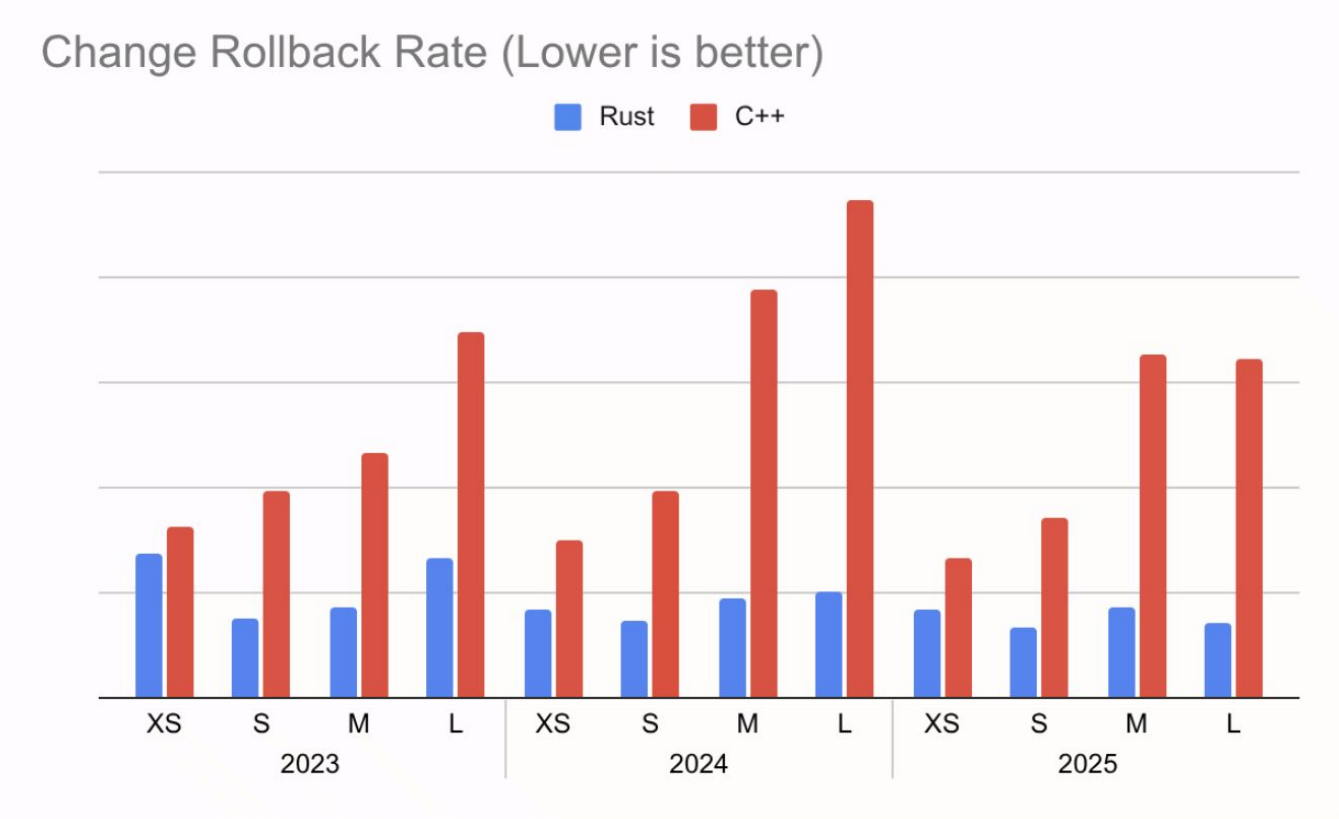
November 13, 2025

We adopted Rust for its security and are seeing **a 1000x reduction in memory safety vulnerability density compared to Android's C and C++ code**. But the biggest surprise was Rust's impact on software delivery. With Rust changes having **a 4x lower rollback rate** and spending **25% less time in code review**, the safer path is now also the faster one.



Net lines of code added: Rust vs. C++, first-party Android code.

This chart focuses on first-party (Google-developed) code (unlike the previous chart that included all first-party and third-party code in Android.) We only include systems languages, C/C++ (which is primarily C++), and Rust.



For medium and large changes, the rollback rate of Rust changes in Android is ~4x lower than C++. This low rollback rate doesn't just indicate stability; it actively improves

Welcome to Comprehensive Rust 🦀

build passing | contributors 335 | stars 33k

This is a free Rust course developed by the Android team at Google. The course covers the full spectrum of Rust, from basic syntax to advanced topics like generics and error handling.

Rust is the latest server-side language at Meta

POSTED ON JANUARY 27, 2026 TO SECURITY & PRIVACY

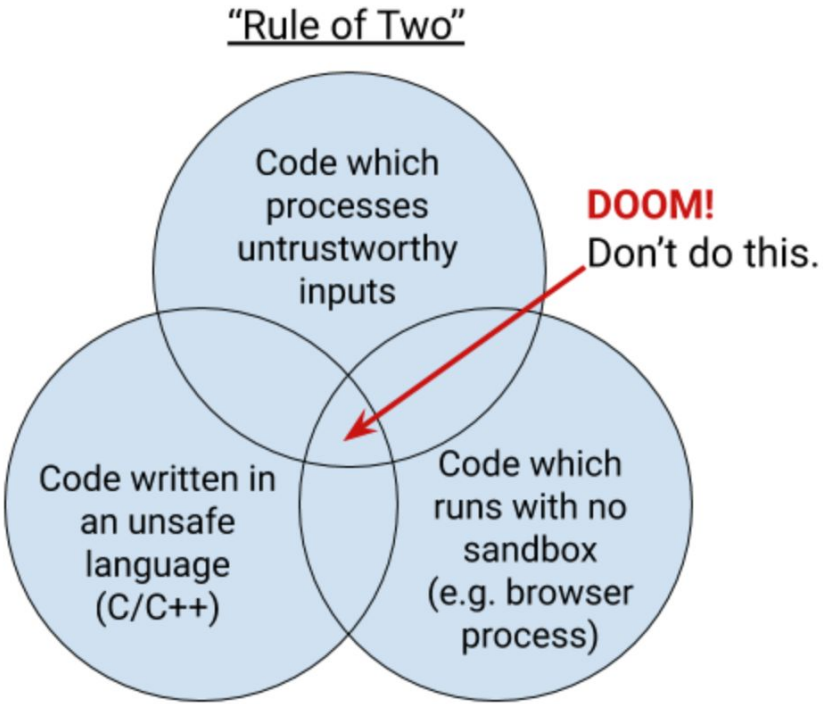
Rust at Scale: An Added Layer of Security for WhatsApp

supported by WhatsApp. WhatsApp made a long-term bet to build that support. In the end, we replaced 160,000 lines of C++ (excluding tests) with 90,000 lines of Rust (including tests). The Rust version showed performance and runtime memory usage advantages over the C++. Given this success, Rust was fully rolled out to all WhatsApp users and many platforms: Android, iOS, Mac, Web, Wearables, and more. With this positive evidence in hand, memory safe languages will play an ever increasing part in WhatsApp's overall approach to application and user security.

The Rule Of 2

When you write code to parse, evaluate, or otherwise handle untrustworthy inputs from the Internet — which is almost everything we do in a web browser! — we like to follow a simple rule to make sure it's safe enough to do so. The Rule Of 2 is: Pick no more than 2 of

- untrustworthy inputs;
- unsafe implementation language; and
- high privilege.



Rewriting the heart of our sync engine

// By Sujay Jayakar • Mar 09, 2020

- We wrote Nucleus in Rust! Rust has been a force multiplier for our team, and betting on Rust was one of the best decisions we made. More than performance, its ergonomics and focus on correctness has helped us tame sync's complexity. We can encode complex invariants about our system in the type system and have the compiler check them for us.
- Almost all of our code runs on a single thread (the "Control thread") and uses Rust's futures library for scheduling many concurrent actions on this single thread. We offload work to other threads only as needed: network IO to an event loop, computationally expensive work like hashing to a thread pool, and filesystem IO to a dedicated thread. This drastically reduces the scope and complexity developers must consider when adding new features.

corrode

Home Learn Blog Podcast Consulting

Rust in Production Podcast

Gama Space with Sebastian Scholz
2026-01-22 S05 E09

Radar with Jeff Kao
2026-01-08 S05 E08

2025 Holiday Special
2025-12-25 S05 E07

Rust for Linux with Danilo Krummrich
2025-12-11 S05 E06

Canonical with Jon Seager
2025-11-27 S05 E05

Roc with Richard Feldman
2025-11-13 S05 E04

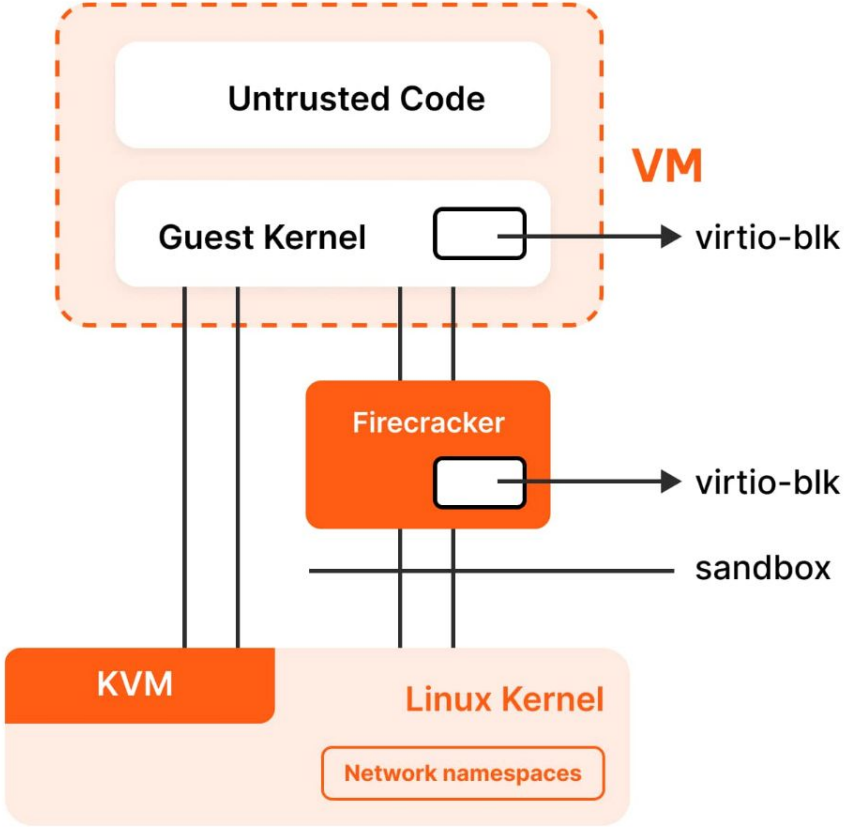
Cloudflare with Edward Wang & Kevin Guthrie
2025-10-30 S05 E03

Scythe Robotics with Andrew Tinka
2025-10-16 S05 E02

Prime Video with Alexandru Ene
2025-10-02 S05 E01

AWS Firecracker

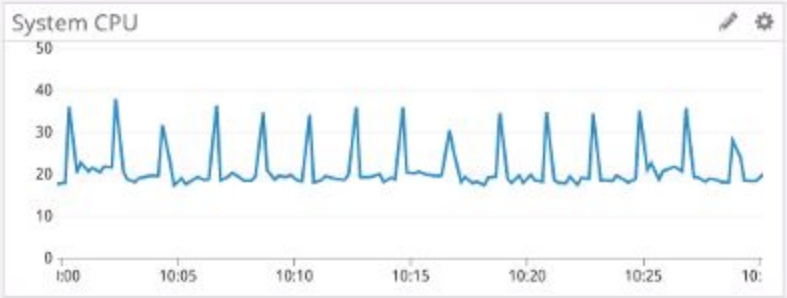
Firecracker

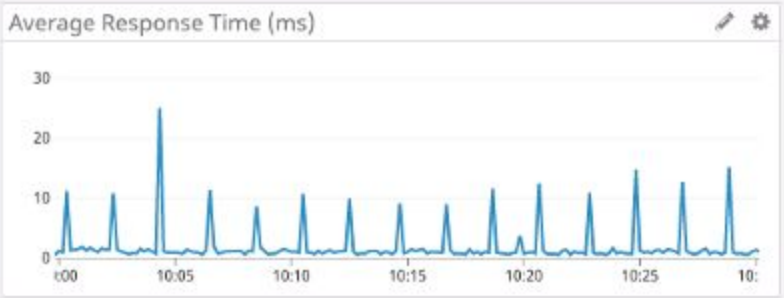


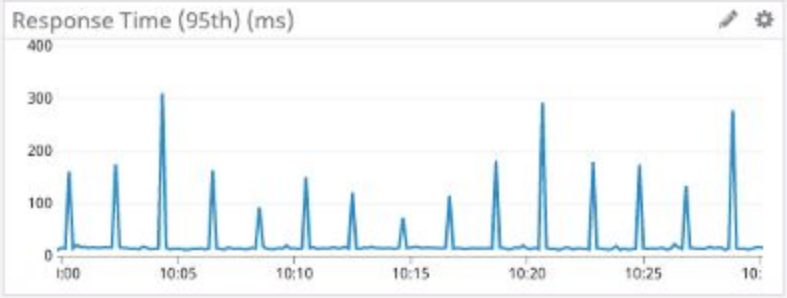
Why Discord is switching from Go to Rust

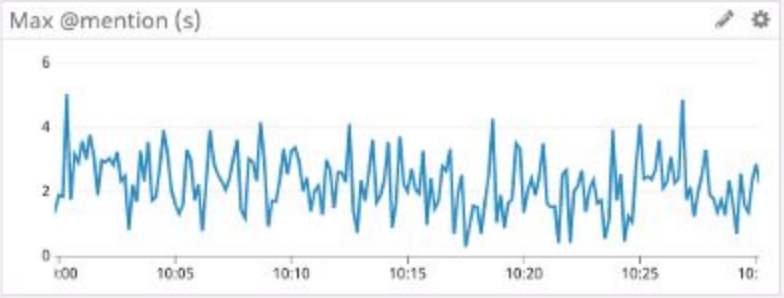
Jesse Howarth

February 4, 2020





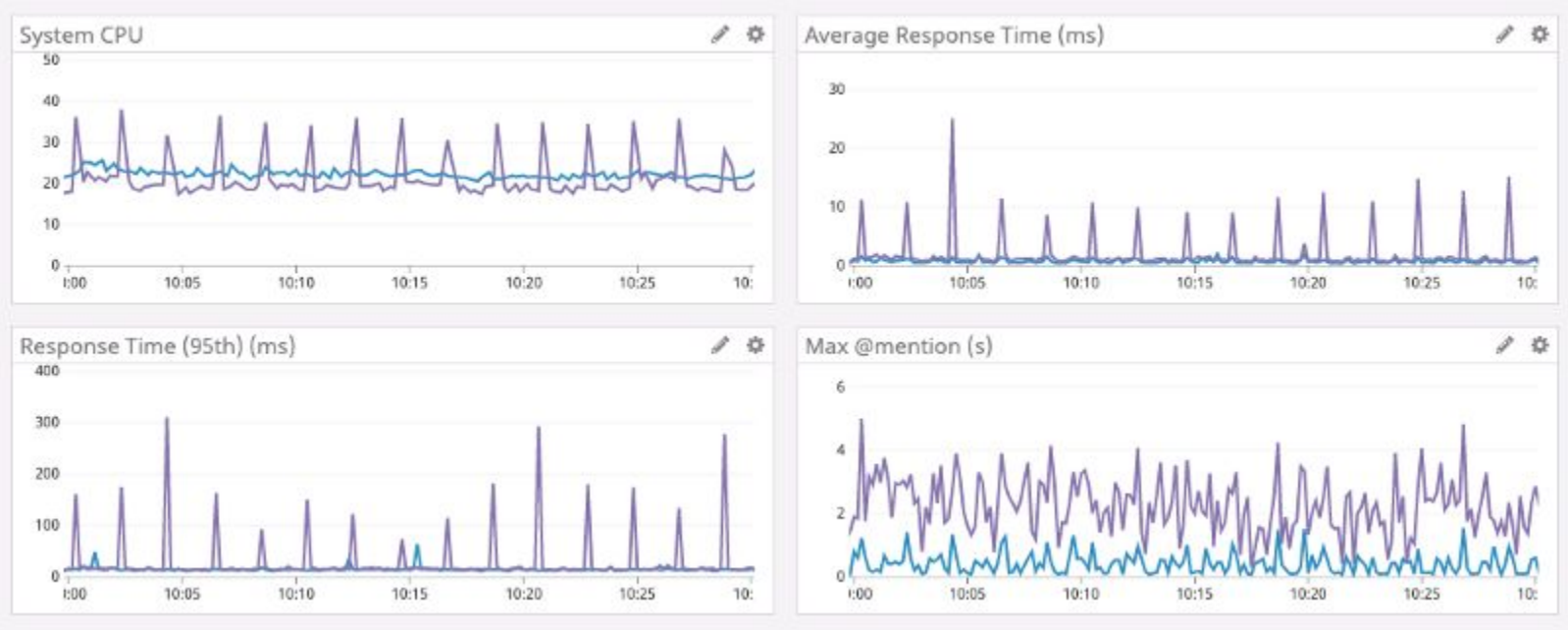




Why Discord is switching from Go to Rust

Jesse Howarth

February 4, 2020



Rust in the Linux Kernel

30M lines of C code

Rust is now an official option in areas such as device drivers (e.g., network drivers, GPU drivers), where integration with existing C APIs is manageable through a binding layer.

The (successful) end of the kernel Rust experiment

[Posted December 10, 2025 by corbet]

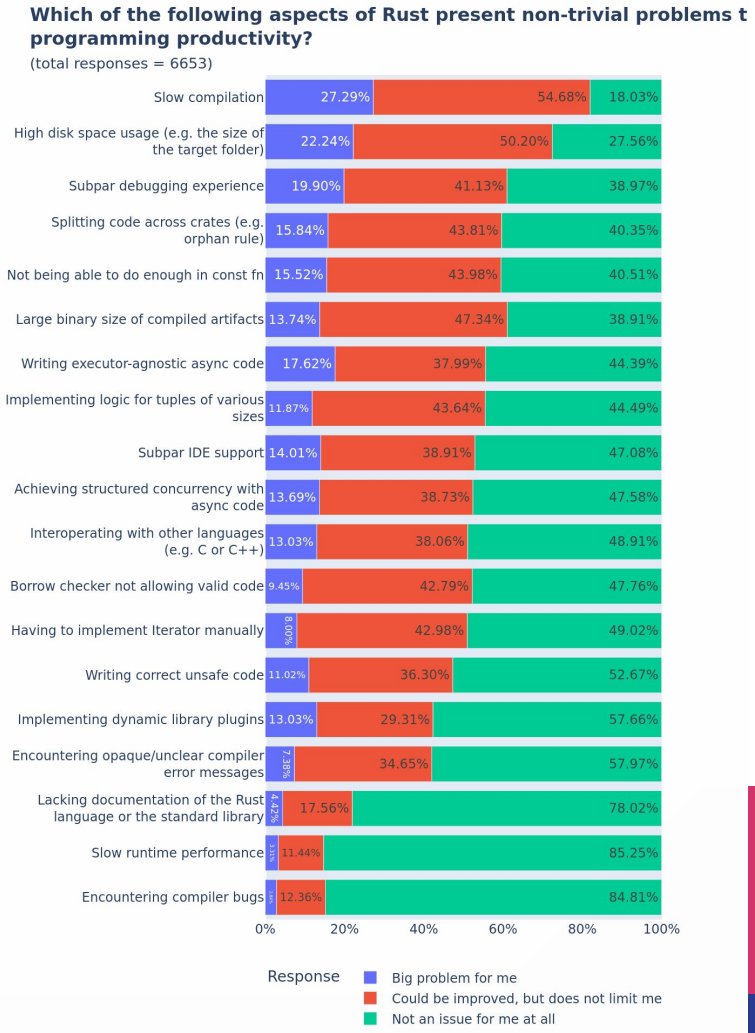
The topic of the Rust experiment was just discussed at the annual Maintainers Summit. The consensus among the assembled developers is that Rust in the kernel is no longer experimental — it is now a core part of the kernel and is here to stay. So the "experimental" tag will be coming off. Congratulations are in order for all of the Rust for Linux team.

One Common Problem

Companies & Linux tend to *really* dislike panics

- Tend to ban usage of .unwrap() and sometimes .expect()
- Prefer Result<T, E>

What is disliked?



What about agentic development?

- In the last few months, agentic LLMs (e.g. Claude Code) have been blowing up
- Why should you still choose Rust?

Joe Savona
@en_JS

React Compiler: Rust edition is coming soon. We've ported the majority of the passes using AI. When the initial port finishes we'll do some updates to get the code in a state we're happy to maintain, then extensive testing and look at performance. More to come soon

10:42 PM · Mar 10, 2026 · **46.7K** Views

Rust in Production: How?

- Most code is already written
- Rust provides an excellent FFI

Rust FFI

Fixed Function Interface

- Describes interoperability capabilities with other languages
- Rust can compile to object files, just like C, and then link!

Rust FFI: Calling Rust from C and vice versa

Rust FFI: C++ Interop

- Real projects have thousands of structs, functions, etc.
- Specific crates are designed to automate the boundary + std type conversion
- https://github.com/dtolnay/cxx/tree/master/demo

Rust in Production: Straight to the source

https://github.com/tokio-rs/mini-redis/tree/master (2 bins)

https://github.com/tokio-rs/mini-redis/blob/master/src/lib.rs (entry)

https://github.com/tokio-rs/mini-redis/blob/master/src/server.rs (run loop)

https://github.com/tokio-rs/mini-redis/blob/master/src/connection.rs (parse)

https://github.com/tokio-rs/mini-redis/blob/master/src/db.rs (Arc<Mutex> db)

https://deepwiki.com/tokio-rs/mini-redis - summary

Quiz on Gradescope (11:14-11:20)

Quiz on Gradescope (11:14-11:20)

a. Performance engineering requires application profiling to discover bottlenecks.
b. Performance engineering requires a program model, to know if you are slower than expected.
c. Padding increases GPU performance because it reduces false sharing.
d. To split up work among GPUs that are far away from each other, we should use tensor parallelism (split each matrix into N pieces and distribute)
e. Data, tensor, and pipeline parallelism can all be combined in both training and inference.

Optional Reading

https://www.howtocodeit.com/guides/writing-production-rust-macros-with-macro-rules

https://www.youtube.com/watch?v=ZbObQYEzZ3Y&list=PL2b0df3jKKiRFEuVNk76ufXagOgEJ9sBZ